

dInference: A Decentralized Marketplace for LLM Inference on Solana

Dhruva Chayapathy, Georgia Institute of Technology

October 2026

Abstract

dInference is a proof-of-concept decentralized marketplace in which users post bounties for large language model (LLM) inference and independent operators compete to serve them. Correctness is enforced by an optimistic rollup: any party may challenge a response within a configurable window, triggering a verification round by a random committee drawn from a per-model verifier pool. Verification uses an **activation fingerprint** — a seeded random projection of the final-layer hidden states produced during generation — which the challenged provider commits on chain and verifiers independently recompute by teacher-forcing the response through the claimed model. The fingerprint runs at native inference speed, in contrast to zkML proofs which impose a 10^3 – $10^6\times$ slowdown. In offline evaluation across 1,500 cross-family samples at a cosine similarity threshold of 0.99, the check achieves 100% true-positive rate on cheating providers (wrong model, no model) and 0% false-positive rate on honest providers. This document describes the protocol, the on-chain program, the off-chain daemons, and the end-to-end flow validated on a local Solana test validator.

Author’s note. This is a personal proof of concept by Dhruva Chayapathy, a student at the Georgia Institute of Technology. Claude (Anthropic) was used as a coding and design-review assistant throughout the implementation. The system has been exercised end-to-end on a local validator; it has not been audited, deployed to a public cluster under adversarial load, or had its economic parameters tuned for mainnet use.

1. Motivation

Running LLM inference as a service currently requires trusting a centralized provider — OpenAI, Anthropic, a hosted-GPU vendor — to actually run the model you asked for, with the sampling parameters you specified, over the full input. There is no mechanism by which a user can verify, after the fact, that the response they received was produced by the claimed model.

Decentralized GPU marketplaces exist, but almost all rely on either (a) reputation and off-chain disputes, (b) redundant execution by K independent nodes (expensive), or (c) zero-knowledge proofs of inference (currently 10^3 – $10^6\times$ slower than native inference, which eliminates throughput as a product). None of these have produced a working at-scale market.

dInference proposes a different point in the design space: **optimistic verification backed by a cheap, model-specific statistical check**. The vast majority of requests settle instantly with no verification work. A small fraction are challenged; those invoke a committee of randomly-selected verifiers who recompute a lightweight fingerprint. Economic rationality — slash the cheater, reward the challenger — keeps the system honest without requiring every request to be re-executed.

2. Protocol overview

The protocol has four actors:

- **User** — posts a bounty for inference on a specific prompt against a specific model.
- **Provider** — runs the model locally; races other providers to claim bounties for models they serve.
- **Verifier** — stakes capital against a particular model; is drafted into verification committees when challenges open.
- **Challenger** — any party (often the user, but anyone may act) who disputes a response by posting a bond.

A single inference request flows through the following stages:

1. **Post.** The user uploads the prompt to Walrus (Sui testnet decentralized storage) and submits `post_bounty(nonce, prompt_txid, bounty_lamports)` on Solana. SOL is escrowed in the global stake vault. The chain emits `BountyPosted`.
2. **Claim.** Every provider daemon subscribed to the model receives the event, fetches the prompt from Walrus, runs inference locally, uploads the response to Walrus, and submits `claim_bounty(response_txid, fingerprint)`. The first valid transaction to land flips the bounty status atomically from `Open` to `Claimed`; losers' transactions revert.
3. **Challenge window.** For a configurable number of slots, anyone may call `challenge(bond)` with a bond proportional to the bounty. The on-chain program deterministically selects K verifiers from the model's verifier pool using `hash(rotating_seed || bounty_pda || i) mod pool.len()` for $i = 0..K$, swap-removing each selected verifier so a single operator cannot sit on two simultaneous committees.
4. **Vote.** Each committee member fetches prompt and response from Walrus, teacher-forces the pair through its local copy of the claimed model, recomputes the fingerprint, computes cosine similarity against the provider's committed fingerprint, and submits `verifier_vote(verdict, fp_cos_scaled)`.
5. **Finalize.** As soon as the running vote tally makes the outcome mathematically decidable (either a quorum of guilty votes is reached, or the honest side has mathematically locked a majority given the remaining uncast votes), anyone may call `finalize_challenge`. Payouts, slashes, and pool re-insertions execute atomically.
6. **Optimistic settlement.** If no challenge opens within the window, anyone may call `finalize_payment` and the full bounty flows to the winning provider.

3. Activation fingerprint

The verification primitive is a seeded random projection of the generated-token hidden states.

Provider side. For each generation step $i = 1..n$, let $h_i \in \mathbb{R}^d$ be the final-layer hidden state at the position that produced the i -th generated token. Let $P \in \mathbb{R}^{d \times k}$ be a Gaussian random matrix seeded from the per-request on-chain rotating seed. The provider commits:

$$f = \sum_{i=1}^n h_i P \in \mathbb{R}^k$$

Verifier side. The verifier reconstructs the full tokenized sequence `[prompt_ids, response_ids]`, runs a single forward pass through the claimed model with `output_hidden_states=True`, extracts the final-layer hidden states at the positions that would have produced each response token (position $p-1$ produces token at position p in a causal LM), and computes f' the same way. The verdict is $\cos(f, f') > \tau$, where $\tau = 0.99$ by default.

Why this works. An honest provider’s hidden-state trajectory under the claimed model matches the verifier’s teacher-forcing exactly (up to float16/bfloat16 numerical noise), so $\cos \approx 1.0$. A cheater running a different model, or no model at all, projects into an unrelated subspace of $\mathbb{R}^k$, so cosine similarity is near zero. Across 1,500 evaluated samples spanning three model families, no honest sample fell below 0.9999 and no cheating sample rose above 0.01.

Why not zkML. Zero-knowledge inference proofs are 3–6 orders of magnitude slower than native inference. For a 1.5B-parameter model generating 500 tokens, that is the difference between seconds and days. The fingerprint check runs at native speed with microseconds of per-token overhead on the provider side and a single forward pass on the verifier side.

Why stop at the activation level. Sampling-level tampering (greedy decoding instead of temperature sampling, slight temperature changes) does not meaningfully reduce the provider’s compute cost — a provider that cheats this way is paying full freight for marginal savings. The attacks that actually save compute (serving a smaller model, serving no model, returning cached junk) manifest as hidden-state divergence and are caught by the fingerprint. Catching sampling-level drift would add false-positive risk for no economic security gain.

4. On-chain program

The smart contract is written in Rust using the Anchor framework and compiled to BPF for Solana. Program ID: `EDX9NGUiJGtGtLDTHbRCqLrb6MbF7ES4eKuxumpfUyWq` (local test deployment).

4.1 Account model

Account	Purpose
<code>GlobalConfig</code>	Admin pubkey, stake minimums, bounty minimums, challenge/vote windows, fingerprint cosine threshold, committee size, quorum fraction.
<code>RotatingSeed</code>	Current 32-byte seed used for committee selection and projection matrices; rotates on a schedule via permissionless <code>rotate_seed</code> .
<code>StakeVault</code>	Single PDA holding all operator stakes and user bounty escrows.
<code>ModelInfo</code>	Per-model: model ID string, hidden dimension, allowed flag.
<code>VerifierPool</code>	Per-model: vector of currently-available verifier operator pubkeys.
<code>ProviderAccount</code>	Per-operator-per-model: stake, reserved stake, activation flag, unstake request state.
<code>VerifierAccount</code>	Per-operator-per-model: stake, activation flag, unstake request state.

Account	Purpose
Bounty	Per-request: user, prompt_txid, bounty_lamports, status, winner, timestamps, response commitment reference.
ResponseCommitment	Per-claim: response_txid, fingerprint vector, submission slot, challenge deadline.
Challenge	Per-challenge: committee, verdicts, status, vote deadline.

4.2 Instructions

Admin: `init_config, init_model`.

Operator lifecycle: `register_provider, register_verifier, request_unstake_*, claim_unstake_*` (two-step with `cooldown ≥ challenge window`).

Bounty lifecycle: `post_bounty, claim_bounty, finalize_payment, reclaim_unclaimed_bounty`.

Challenge: `challenge, verifier_vote, finalize_challenge`.

Housekeeping: `rotate_seed`.

4.3 Committee selection

Given a verifier pool P and target committee size K , selection runs K iterations of:

```
idx_i = u64::from_le_bytes(hash(rotating_seed || bounty_pda || i)[0..8]) % P.len()
committee.push(P.swap_remove(idx_i))
```

The algorithm is $O(K)$ hashes and $O(K)$ swap-removes. It is deterministic — SDKs can replay the hash chain to predict committee membership before the on-chain transaction lands. Swap-remove ensures a verifier cannot simultaneously serve two committees. After `finalize_challenge`, surviving (not-slashed, still-active) committee members are re-inserted into the pool.

4.4 Early-finalize decidability

Let n = committee size, g = guilty votes cast, h = honest votes cast, $r = n - g - h$ = remaining uncast. Let $q = \lfloor n \cdot \text{quorum_fraction} / 10_000 \rfloor$. The challenge is **decidable** iff:

- $g \geq q$ (guilty quorum reached — slash the provider), or
- $g + r < q$ (not enough remaining votes to reach quorum — exonerate).

`finalize_challenge` is permissionless once decidable, so verifiers (or anyone) can resolve immediately when the tally locks, rather than waiting for the full window.

4.5 Stake invariants

Provider stake must satisfy, at claim time:

$$\text{stake} \geq 2 \cdot (0.25 \cdot \text{bounty} + \text{verify_pool})$$

This guarantees that a half-stake slash always covers both the challenger reward ($1.25\times$ bounty on success) and the verify fees paid out to the committee. The provider's `reserved_stake` field is incremented at claim and decremented at finalize to prevent double-booking across concurrent claims.

Verifier stake must satisfy `stake ≥ dishonesty_fee` at registration so a single slash cannot underflow the account.

5. Off-chain components

The off-chain stack is Python 3.11 using `solders` for transaction building, `solana-py` for RPC, `transformers` for model execution, and `httpx` for Walrus I/O.

5.1 Fingerprint pipelines

Both the provider and verifier wrap their logic as a `transformers.Pipeline` subclass. The provider pipeline runs `model.generate` with hidden-state capture, projects the per-step hidden states through the seeded matrix, and emits `{text, fingerprint, seed}`. The verifier pipeline teacher-forces the full tokenized sequence through a single forward pass, extracts the hidden states at response positions, recomputes the projection, and emits `{fp_cos, passed, truncated, reason}`.

Models are declared in `config/models.yaml` with an explicit `type: instruct | base` field. Instruct models apply the tokenizer's chat template with `add_generation_prompt=True`; base models tokenize the raw prompt. The verifier mirrors this exactly, otherwise prompt tokenization drifts and the fingerprint comparison collapses to noise.

The provider decodes the generated token ids with `skip_special_tokens=False` so the trailing stop marker (`<|im_end|>`, EOS, etc.) survives through Walrus to the verifier, where it is needed for the completion-rule check. The user-facing SDK strips special tokens for display.

5.2 Daemons

Provider daemon (`off_chain_inference/ramp.py`) subscribes to `logsSubscribe` filtered by program ID, parses `Program data:` lines against known event discriminators, and on each `BountyPosted` for its model: fetches prompt from Walrus → runs inference → uploads response to Walrus → submits `claim_bounty`. First-come-first-served with a single-slot busy flag.

Verifier daemon (`off_chain_verifier/ramp.py`) subscribes to the same stream, and on each `ChallengeOpened` whose committee contains its operator pubkey: fetches both blobs → runs the fingerprint check → submits `verifier_vote` → opportunistically calls `finalize_challenge` if the current tally is decidable. A failed Walrus fetch auto-votes guilty on the theory that the provider failed to post what they claimed.

5.3 User SDK

`DInferenceClient.inference(prompt, model_id)` uploads the prompt to Walrus, submits `post_bounty`, polls for the response commitment PDA, fetches the response from Walrus, strips special tokens, and returns `(bounty_pda, text)`. `.challenge(bounty_pda, model_id)` posts a bond and opens a challenge.

6. Storage layer

Walrus (Sui testnet) was selected over Arweave after library-compatibility issues with `arweave-python-client` on locally-generated JWKs, and after measuring 60-second indexing lag on the Turbo-SDK-based Irys alternative. Walrus publishes return a content-addressed blob ID (sha256 of content, base64url) immediately and the aggregator serves the blob within 1–2 seconds, with no indexing lag. Measured latencies on the test deployment:

- Upload (prompt or response): 3–9 seconds cold, 3–6 seconds warm.
- Fetch (aggregator GET): 1–2 seconds.

Content-addressing means blob IDs can be committed on-chain before the blob is retrievable, and verifiers can independently confirm that what they fetched is what the provider committed.

7. Attack surface

Attack	Caught by
Run a different model	Fingerprint check; $\cos \approx 0$.
Don't run the model at all	Fingerprint check; no valid hidden states to project.
Truncate response to save compute	Verifier checks last token is a declared stop token if $n_tokens < max_new_tokens$.
Sampling tampering (greedy, temperature)	Not caught by design; attack is economically irrational.
Provider wins race, then stalls	Challenge window + <code>claim_bounty</code> reverts on re-attempt.
Challenger spams fake challenges	Loses bond on each wrong outcome.
Verifier colludes to vote guilty	Minority voters slashed; fee is constant regardless of verdict to remove directional incentive.
Verifier doesn't vote	Treated identically to a minority vote (slashed).
Two providers claim the same bounty	Atomic status flip; second claim reverts.
Provider under-stakes relative to bounty	<code>claim_bounty</code> enforces $stake \geq 2 \cdot (0.25 \cdot bounty + verify_pool)$.
MEV/pre-reveal of committee membership	Committee hashes include the rotating seed, which rotates faster than the challenge window.

8. Evaluation

Offline evaluation ran $500 \text{ prompts} \times 3 \text{ scenarios}$ (1,500 samples total) on Modal with A10G GPUs.

Scenario	Actual model served	Mean fp_cos	Pass rate
Honest	Qwen2.5-1.5B-Instruct	1.0000	1.0000
Cheat (smaller same-family model)	Qwen2.5-0.5B-Instruct	-0.0059	0.0000
Cheat (cross-family similar-size)	SmolLM2-1.7B-Instruct	-0.0018	0.0000

At `cos_threshold = 0.99`: FPR = 0.0000, TPR = 1.0000, accuracy = 1.0000.

Live local-validator end-to-end runs reproduce the honest-flow result: `fp_cos = 1.0000` on challenged honest responses, no slashing, provider stake and verifier stakes intact after acquittal.

9. Limitations and open work

Economic parameters are placeholders. The live `GlobalConfig` has `verify_fee_lamports = 0`, `dishonesty_fee_lamports = 0`, `challenger_reward_bps = 0`, and both time windows near U64 max. These need realistic values before any adversarial deployment. The mechanism itself is wired through — reserved stake correctly tracks claim-time escrow, slashing arithmetic preserves vault invariants, pool reinsertion is idempotent — but the knobs are untuned.

Fingerprint reconstruction is fragile at the tokenization boundary. The verifier reconstructs the provider’s full token sequence by concatenating `chat_template(prompt) + tokenize(response, add_special_tokens=False)`. BPE can re-merge tokens differently at the join point, which does not seem to matter in practice (`cos` is still 1.0000 to four decimals) but is not guaranteed. The clean fix is for the provider to upload raw generated token IDs rather than decoded text, and have the verifier teacher-force directly on those IDs.

Fingerprint determinism depends on dtype. Hidden-state computations in `bfloat16` are not bitwise-identical across GPU architectures or driver versions. The 0.99 threshold has significant slack for this but it has not been systematically characterized at the limit (e.g., `fp16` vs `bf16`, consumer GPU vs datacenter GPU).

Walrus is testnet storage. Walrus mainnet is live but the fingerprint pipeline currently targets the publisher/aggregator endpoints for Sui testnet. Switching endpoints is a one-line change but mainnet pricing, retention guarantees, and SLA have not been evaluated.

No prompt privacy. Prompt and response are stored in cleartext on public decentralized storage. For sensitive use cases, an encryption layer (client-side encryption with keys derived from the user’s keypair) would be required; the chain would hold the ciphertext hash, the user would share the decryption key with the provider via some out-of-band or on-chain-encrypted channel, and verifiers would need the same key. This is unimplemented.

Only Solana is targeted. The protocol is not chain-specific in principle — the committee-selection and slashing logic port to any environment with cheap deterministic hashing and native transfers. No other chain has been attempted.

10. Acknowledgements

Claude (Anthropic) was used as a coding and design-review assistant throughout the implementation of the smart contract, off-chain daemons, SDK, and this document. All design decisions, protocol choices, and the final shape of the system are the author’s; Claude contributed code, patterns, and critique against the author’s direction.

The activation-fingerprint idea builds on the general observation that LLM hidden-state trajectories are highly model-specific and that random projections preserve enough of this structure to serve as a cheap verification signal. Related ideas appear in prior work on model watermarking and output attribution; dInference’s contribution is pairing this check with an on-chain optimistic rollup to make a working economic market.

11. References

Code and reproducibility artifacts are available at the project repository. The smart contract (`programs/difference/`), off-chain pipelines (`off_chain_inference/`, `off_chain_verifier/`), chain client (`chain/`), user SDK (`sdk/`), and end-to-end orchestration script (`scripts/local_e2e.sh`) together constitute roughly 4,000 lines of Rust and Python.

The evaluation harness (`scripts/experiment_cross_family.py`) runs on Modal and produces the TPR/FPR numbers in §8.